Since we've already verified that RHEL, /boot, /boot/efi, /home, and swap are all on **mmcblk0**, you can erase the old NVMe layout and turn the whole NVMe into a fresh encrypted /data.

Everything on /dev/nvme0n1 will be destroyed. Based on the output you just provided, that is the intended disk.

I recommend one encrypted partition spanning essentially the entire NVMe:

/dev/nvme0n1

└─ /dev/nvme0n1p1

└─ LUKS

└─ /dev/mapper/crypt_data

└─ XFS

└─ /data

1. Make absolutely sure nothing from NVMe is mounted

Bash

findmnt | grep nvme

sudo pvs

findmnt | grep nvme should produce no output, and pvs should continue to show only:

/dev/mmcblk0p3 rhel

2. Remove the old NVMe partition/filesystem signatures

First inspect one last time:

Bash

sudo lsblk -f /dev/nvme0n1

Then erase the old signatures:

Bash

sudo wipefs -a /dev/nvme0n1

Because the old partitions may still be known to the kernel, refresh the partition table:

Bash

```
sudo partprobe /dev/nvme0n1
```

Check:

Bash

```
lsblk /dev/nvme0n1
```

We ultimately want just:

nvme0n1 1.8T disk

If old child partitions remain displayed at this point, **stop and show me the output rather than continuing**.

3. Create a new GPT and one large partition

Use parted:

Bash

```
sudo parted -s /dev/nvme0n1 mklabel gpt
sudo parted -s /dev/nvme0n1 mkpart primary 1MiB 100%
```

Tell the kernel to reread it:

Bash

```
sudo partprobe /dev/nvme0n1
```

Verify:

Bash

```
lsblk /dev/nvme0n1
```

You should now have approximately:

```
nvme0n1 1.8T
└─nvme0n1p1 1.8T
```

4. Encrypt the new partition with LUKS

Now:

Bash

```
sudo cryptsetup luksFormat /dev/nvme0n1p1
```

It will warn you that the operation is destructive and ask for confirmation. Then choose a strong LUKS passphrase.

Open it:

Bash

```
sudo cryptsetup open /dev/nvme0n1p1 crypt_data
```

Verify:

Bash

```
lsblk -f
```

You should now have:

```
nvme0n1
└─nvme0n1p1 crypto_LUKS
└─crypt_data
```

5. Create the /data filesystem

I recommend XFS because that is RHEL's standard filesystem:

Bash

```
sudo mkfs.xfs /dev/mapper/crypt_data
```

Create the mount point:

Bash

```
sudo mkdir -p /data
```

Mount it:

Bash

sudo mount /dev/mapper/crypt_data /data

Verify:

Bash

df -h /data

You should see approximately 1.8 TB:

Filesystem Size Used Avail Use% Mounted on

/dev/mapper/crypt_data 1.8T ... ... /data

6. Make /data usable by osi

If this disk is primarily your data area:

Bash

sudo chown osi:osi /data

Test:

Bash

touch /data/testfile

ls -l /data/testfile

rm /data/testfile

7. Configure automatic unlocking during boot

First obtain the LUKS UUID:

Bash

sudo cryptsetup luksUUID /dev/nvme0n1p1

Suppose it returns:

12345678-1234-1234-1234-123456789abc

Add this to /etc/crypttab:

crypt_data UUID=12345678-1234-1234-1234-123456789abc none luks

Then get the XFS UUID:

Bash

sudo blkid /dev/mapper/crypt_data

Suppose that returns an XFS UUID of:

aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee

Add to /etc/fstab:

UUID=aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee /data xfs defaults 0 0

With that configuration, the boot process becomes:

QSPI

↓

eMMC

↓

RHEL boots

↓

LUKS asks for NVMe passphrase

↓

crypt_data opens

↓

/data mounts

I recommend **stopping after Step 5 initially**. Confirm that /data mounts and works correctly before modifying /etc/crypttab and /etc/fstab. That avoids turning a simple storage configuration issue into a boot-time problem.